

- Final Client Round — Banking Domain Questionnaire (Part 2)
 - Section 2: Terraform & IaC for Banking (5 Questions)
 - Q11. How do you structure Terraform modules for a bank with multiple applications across multiple environments?
 - Q12. How do you manage Terraform state securely for a bank?
 - Q13. How do you handle Terraform drift detection in a banking environment where unauthorized changes are a compliance violation?
 - Q14. How do you handle secrets in Terraform for banking without exposing them in state?
 - Q15. How would you implement compliance-as-code for banking regulations using Terraform?
 - Section 3: CI/CD for Banking (5 Questions)
 - Q16. How do you design a CI/CD pipeline for a banking application that requires change management approval before production deployment?
 - Q17. How do you implement blue-green deployments for a banking application on EKS?
 - Q18. How do you handle rollback in a banking production environment?
 - Q19. How do you manage container image security for a banking application?
 - Q20. How do you ensure CI/CD pipeline security itself in a banking context?
 - Section 4: EKS/Kubernetes for Banking (5 Questions)
 - Q21. How do you design EKS for multi-tenant banking microservices?
 - Q22. How do you implement IRSA (IAM Roles for Service Accounts) on EKS and why is it critical for banking?
 - Q23. How do you handle EKS upgrades in a banking environment with zero downtime?
 - Q24. How do you implement network policies on EKS for a banking application?
 - Q25. How do you handle pod security in a banking EKS cluster?

Final Client Round — Banking Domain Questionnaire (Part 2)

Candidate: Pushparaj Naik — AWS Cloud Architect (22+ years)

Context: Final round with banking-sector client

Section 2: Terraform & IaC for Banking (5 Questions)

Q11. How do you structure Terraform modules for a bank with multiple applications across multiple environments?

Answer:

I follow a **layered module architecture** with strict separation:

```
terraform/
├── modules/                                # Reusable building blocks
│   ├── networking/                        # VPC, subnets, TGW attachments
│   ├── compute/                          # EKS, ECS, EC2 patterns
│   ├── database/                         # RDS, DynamoDB, ElastiCache
│   ├── security/                         # KMS, WAF, GuardDuty, Config Rules
│   ├── storage/                          # S3 with compliance policies
│   ├── observability/                   # CloudWatch, alarms, dashboards
│   └── compliance/                      # SCPs, Config conformance packs
├── compositions/                         # Application-specific stacks
│   ├── core-banking/                    # Composes modules for core banking
│   ├── internet-banking/                # Composes modules for IB
│   └── data-platform/                   # Composes modules for analytics
├── environments/
│   ├── dev.tfvars
│   ├── uat.tfvars
│   └── prod.tfvars
└── platform/                             # Account-level baseline
    ├── landing-zone/
    └── shared-services/
```

Key Principles for Banking:

1. **Compliance baked into modules:** Every S3 module enforces encryption, versioning, public access blocks by default — developers can't skip it
2. **No inline policies:** All IAM is managed through the security module

3. **Tagging enforcement:** Every module requires `data_classification`, `regulatory_scope`, `cost_center` tags

4. **Variable validation:**

```
variable "data_classification" {
  type = string
  validation {
    condition      = contains(["public", "internal", "confidential",
"restricted"], var.data_classification)
    error_message = "Must be: public, internal, confidential, or
restricted."
  }
}
```

This is the same pattern I built at Rio Tinto — modular Terraform stack with reusable patterns for S3, RDS, Glue, IAM, and KMS that teams could adopt while remaining production-ready.

Q12. How do you manage Terraform state securely for a bank?

Answer:

State files contain sensitive data (resource IDs, connection strings, sometimes outputs with secrets). For banking:

State Backend:

```
terraform {
  backend "s3" {
    bucket      = "bank-terraform-state-${account_id}"
    key         = "${application}/${environment}/terraform.tfstate"
    region      = "ap-south-1"
    dynamodb_table = "terraform-lock"
    encrypt     = true
    kms_key_id   = "alias/terraform-state-key"
  }
}
```

Security Controls:

Control	Implementation
Encryption	S3 SSE-KMS with dedicated CMK (not default key)
Access control	Bucket policy restricts to specific IAM roles per environment
Locking	DynamoDB table prevents concurrent modifications
Versioning	S3 versioning enabled — rollback to previous state
Audit	S3 access logging + CloudTrail data events on state bucket
Isolation	Separate state file per application per environment
No local state	CI/CD only — terraform apply never runs from laptops
State inspection	terraform state list/show requires elevated IAM role

State Isolation Strategy:

```
s3://bank-terraform-state/  
├─ core-banking/prod/terraform.tfstate  
├─ core-banking/uat/terraform.tfstate  
├─ internet-banking/prod/terraform.tfstate  
├─ data-platform/prod/terraform.tfstate  
└─ platform/shared-services/terraform.tfstate
```

At Rio Tinto, I implemented this exact pattern — state isolation per environment with OIDC-based cross-account roles so GitHub Actions runners never have persistent credentials.

Q13. How do you handle Terraform drift detection in a banking environment where unauthorized changes are a compliance violation?

Answer:

Drift in banking is a **compliance event**, not just an operational issue. My approach:

Detection:

1. **Scheduled terraform plan** via GitHub Actions (every 4 hours):

```
- name: Drift Detection
  run: |
    terraform plan -detailed-exitcode -var-file=environments/prod.tfvars
    # Exit code 2 = drift detected
  continue-on-error: true

- name: Alert on Drift
  if: steps.plan.outcome == 'failure'
  run: |
    aws sns publish --topic-arn $DRIFT_ALERT_TOPIC \
      --message "Terraform drift detected in production"
```

2. **AWS Config Rules:** Detect changes not made through Terraform (e.g., someone modifying a security group via console)

3. **CloudTrail Alerting:** EventBridge rule on **ModifySecurityGroup**, **PutBucketPolicy** etc. from console (not from CI/CD role)

Response:

- **Auto-remediate** non-critical drift (tags, descriptions)
- **Alert + ticket** for security-relevant drift (SG rules, IAM policies, encryption settings)
- **Incident process** for unauthorized changes in PCI scope

Prevention:

- SCPs deny **ec2:***, **rds:*** etc. for human IAM roles in production — only CI/CD role can modify infrastructure
- AWS Config rule: **required-tags** — resources without Terraform tags are flagged as non-compliant

Q14. How do you handle secrets in Terraform for banking without exposing them in state?

Answer:

This is critical — Terraform state can contain secret values in plaintext.

Strategy: Generate, Don't Store

```

# Generate password – stored in state (encrypted via KMS backend)
resource "random_password" "db_master" {
  length  = 32
  special = true
}

# Store in Secrets Manager – application reads from here, not Terraform
resource "aws_secretsmanager_secret_version" "db_master" {
  secret_id      = aws_secretsmanager_secret.db_creds.id
  secret_string = jsonencode({
    username = "admin"
    password = random_password.db_master.result
  })

  # Prevent Terraform from showing the value
  lifecycle {
    ignore_changes = [secret_string]
  }
}

# RDS reads from Secrets Manager
resource "aws_rds_cluster" "main" {
  master_username      = "admin"
  manage_master_user_password = true # AWS manages the password
  master_user_secret_kms_key_id = aws_kms_key.db.arn
}

```

Best Approach for RDS (AWS-managed):

- Use `manage_master_user_password = true` — AWS generates, stores, and rotates the password in Secrets Manager automatically
- Terraform never sees the password, state never contains it

Additional Controls:

- Mark sensitive outputs: `sensitive = true`
- Never use `terraform output` in CI/CD logs for sensitive values
- State file encrypted with KMS CMK
- `.tfvars` files with secrets never committed — use environment variables or Secrets Manager data sources

Q15. How would you implement compliance-as-code for banking regulations using Terraform?

Answer:

I implement compliance at **three layers**:

Layer 1: Preventive (Terraform Module Defaults)

```
# Every S3 bucket module enforces banking compliance
module "s3_bucket" {
  # These are NOT optional – hardcoded in the module
  versioning_enabled      = true
  server_side_encryption  = "aws:kms"
  block_public_acls       = true
  block_public_policy     = true
  object_lock_enabled     = var.regulatory_scope == "pci" ? true : false
  access_logging_enabled  = true
  lifecycle_glacier_days  = 90
  lifecycle_expiration_years = 7
}
```

Layer 2: Detective (AWS Config + Security Hub)

```
resource "aws_config_conformance_pack" "pci_dss" {
  name = "PCI-DSS-Conformance-Pack"
  template_body = file("${path.module}/conformance-packs/pci-dss.yaml")
}

resource "aws_securityhub_standards_subscription" "pci" {
  standards_arn = "arn:aws:securityhub:ap-south-1::standards/pci-dss/v/3.2.1"
}
```

Layer 3: Corrective (Auto-Remediation)

```
# Auto-remediate unencrypted S3 buckets
resource "aws_config_remediation_configuration" "s3_encryption" {
  config_rule_name = "s3-bucket-server-side-encryption-enabled"
  target_type      = "SSM_DOCUMENT"
  target_id        = "AWS-EnableS3BucketEncryption"
  automatic        = true

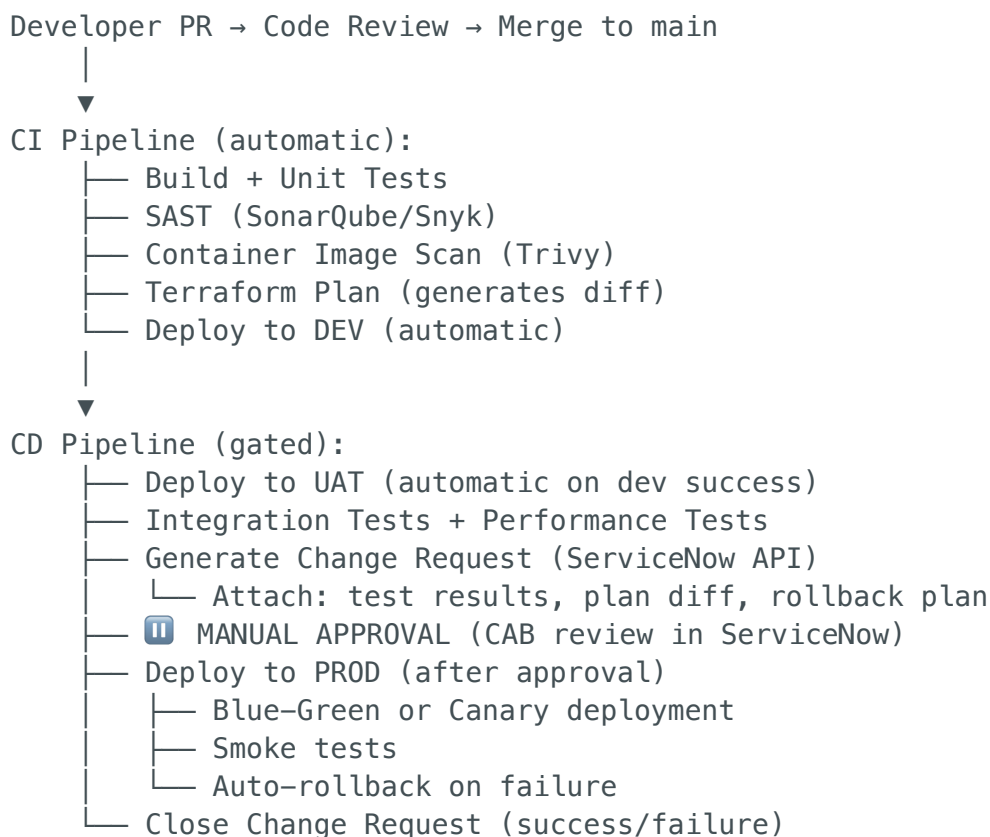
  parameter {
    name          = "BucketName"
    resource_value = "RESOURCE_ID"
  }
}
```

Section 3: CI/CD for Banking (5 Questions)

Q16. How do you design a CI/CD pipeline for a banking application that requires change management approval before production deployment?

Answer:

Banking CI/CD must integrate with **ITIL change management** processes:



GitHub Actions Implementation:

```
deploy-prod:
  needs: [deploy-uat, security-scan]
  environment: production # Requires manual approval in GitHub
  steps:
```



```
- name: Create Change Request
  run: |
    CR_NUMBER=$(python scripts/create_change_request.py \
      --summary "Deploy ${github.sha}" \
      --risk-level "standard")
    echo "CR_NUMBER=$CR_NUMBER" >> $GITHUB_ENV

- name: Terraform Apply
  run: terraform apply -auto-approve -var-file=environments/prod.tfvars

- name: Close Change Request
  if: always()
  run: python scripts/close_change_request.py --cr $CR_NUMBER --status
${{ job.status }}
```

At Rio Tinto, I implemented this with GitHub Actions + OIDC + manual approvals for production — the same pattern scales for banking with the addition of ServiceNow integration.

Q17. How do you implement blue-green deployments for a banking application on EKS?

Answer:

For banking, blue-green provides **instant rollback** — critical for payment processing:

Architecture:

```
Route 53 (weighted routing)
├── 100% → ALB-Blue (current production - v1.2.3)
│       └── EKS Target Group (blue pods)
└── 0%  → ALB-Green (new version - v1.2.4)
       └── EKS Target Group (green pods)
```

Deployment Steps:

1. Deploy new version to green namespace: `kubectl apply -n green`
2. Run automated smoke tests against green ALB endpoint
3. Run synthetic transaction tests (test accounts, not real money)
4. Shift 5% traffic to green (canary validation)
5. Monitor error rates, latency, transaction success rate for 15 minutes

6. If healthy → shift 100% to green
7. If unhealthy → shift 100% back to blue (< 30 seconds)
8. Keep blue running for 24 hours as rollback safety net

Key Banking Considerations:

- **Database compatibility:** Both blue and green must work with the same DB schema — requires backward-compatible migrations only
 - **Session management:** Use external session store (ElastiCache) so sessions survive the switch
 - **Transaction integrity:** Implement idempotency keys so duplicate processing during switch doesn't cause double charges
 - **Audit trail:** Log every traffic shift with who/when/why in CloudTrail
-

Q18. How do you handle rollback in a banking production environment?

Answer:

Banking rollback must be **fast, safe, and auditable**.

Application Rollback (EKS):

```
# Instant rollback using Kubernetes
kubectl rollout undo deployment/payment-service -n production

# Or roll back to specific revision
kubectl rollout undo deployment/payment-service --to-revision=3
```

Infrastructure Rollback (Terraform):

```
# Option 1: Revert the commit and re-apply
git revert HEAD
terraform apply -var-file=environments/prod.tfvars

# Option 2: Use previous state (versioned S3)
# List state versions
aws s3api list-object-versions --bucket state-bucket --prefix
prod/terraform.tfstate
# Restore previous version
```

Database Rollback:

- Forward-only migrations with backward compatibility
- Never drop columns in same release as code change
- Point-in-time restore capability (RDS: 5-minute granularity)
- Aurora clone for testing rollback before executing

Automated Rollback Triggers:

- Error rate > 1% for 5 minutes
- P99 latency > 2x baseline
- Transaction success rate < 99.5%
- Any 5xx response on payment endpoints

Q19. How do you manage container image security for a banking application?

Answer:

Image Pipeline:



Controls:

Stage	Tool	Policy
Base image	Only approved base images from internal ECR	No latest tag, pinned versions only
Build	Multi-stage Dockerfile, non-root user	No secrets in build args
Scan	Trivy + ECR native scanning	Block deploy if CRITICAL CVEs
Sign	Notation (Notary v2) / cosign	Only signed images can deploy
Store	ECR with immutable tags	Tag protection, lifecycle policies

Stage	Tool	Policy
Deploy	EKS admission controller (OPA/Kyverno)	Reject unsigned or unscanned images
Runtime	Falco	Detect anomalous container behavior

EKS Admission Policy (Kyverno):

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-image-from-ecr
spec:
  rules:
    - name: only-ecr-images
      match:
        resources:
          kinds: ["Pod"]
      validate:
        message: "Images must come from approved ECR repository"
        pattern:
          spec:
            containers:
              - image: "*.dkr.ecr.ap-south-1.amazonaws.com/*"
```

Q20. How do you ensure CI/CD pipeline security itself in a banking context?

Answer:

The pipeline is a **high-value attack target** — if compromised, attackers can deploy malicious code to production.

Pipeline Security Controls:

1. No Long-Lived Credentials:

- OIDC federation for GitHub Actions → AWS (no stored access keys)
- Pipeline assumes role with session duration of 1 hour max
- Implemented this at Rio Tinto — removed all long-lived AWS credentials

2. Branch Protection:

- `main` branch: require 2 approvals, CODEOWNERS review, signed commits
- No force push, no branch deletion
- Status checks must pass (build, test, scan)

3. Least-Privilege Pipeline Roles:

- Separate IAM roles per environment (dev-deployer, prod-deployer)
- Prod role has MFA condition or requires GitHub environment approval
- No `*` actions — explicit permissions only

4. Artifact Integrity:

- Container images signed with cosign
- Terraform providers pinned with checksums in `.terraform.lock.hcl`
- Dependency scanning (Dependabot, Snyk)

5. Audit:

- Every pipeline run logged with: who triggered, what changed, approval chain
- CloudTrail captures all API calls made by pipeline IAM role
- Pipeline logs retained for 1 year (compliance)

Section 4: EKS/Kubernetes for Banking (5 Questions)

Q21. How do you design EKS for multi-tenant banking microservices?

Answer:

Namespace-based isolation with security boundaries:

```
EKS Cluster
├─ Namespace: payments      (PCI scope – strict isolation)
├─ Namespace: accounts      (core banking)
```

```
└─ Namespace: channels      (internet/mobile banking)
└─ Namespace: monitoring    (observability stack)
└─ Namespace: istio-system  (service mesh)
```

Isolation Mechanisms:

Layer	Control
Namespace	ResourceQuotas, LimitRanges per namespace
Network	NetworkPolicies: default deny all, explicit allow
IAM	IRSA: each namespace's service accounts get scoped IAM roles
RBAC	Kubernetes RBAC: teams can only access their namespace
Pod Security	Pod Security Standards: restricted profile for PCI namespace
Secrets	Secrets Store CSI Driver → AWS Secrets Manager (not K8s secrets)
Service Mesh	Istio mTLS between all services, authorization policies

Q22. How do you implement IRSA (IAM Roles for Service Accounts) on EKS and why is it critical for banking?

Answer:

IRSA provides **pod-level AWS credential scoping** — critical because without it, all pods on a node share the same IAM permissions.

Why it matters for banking:

- Without IRSA: Payment pod and logging pod share the same EC2 instance role → logging pod compromise = payment data access
- With IRSA: Payment pod has **payment-role** (DynamoDB access), logging pod has **logging-role** (CloudWatch only)

Implementation:

```
# 1. OIDC Provider
resource "aws_iam_openid_connect_provider" "eks" {
```

```

url            = aws_eks_cluster.main.identity[0].oidc[0].issuer
client_id_list = ["sts.amazonaws.com"]
thumbprint_list =
[data.tls_certificate.eks.certificates[0].sha1_fingerprint]
}

# 2. IAM Role with trust policy scoped to specific service account
resource "aws_iam_role" "payment_service" {
  name = "payment-service-role"
  assume_role_policy = jsonencode({
    Statement = [{
      Effect = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.eks.arn }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${replace(aws_eks_cluster.main.identity[0].oidc[0].issuer,
"https:///", "")}:sub" = "system:serviceaccount:payments:payment-sa"
        }
      }
    }]
  })
}

# 3. Kubernetes Service Account annotation
resource "kubernetes_service_account" "payment" {
  metadata {
    name      = "payment-sa"
    namespace = "payments"
    annotations = {
      "eks.amazonaws.com/role-arn" = aws_iam_role.payment_service.arn
    }
  }
}

```

This ensures the payment service can only access its specific DynamoDB table and KMS key — not another service's resources.

Q23. How do you handle EKS upgrades in a banking environment with zero downtime?

Answer:

EKS upgrades are high-risk in banking — I follow a **controlled, staged process**:

Phase 1: Preparation (1-2 weeks before)

- Review Kubernetes changelog for breaking changes and deprecated APIs

- Run `pluto detect-all-in-cluster` to find deprecated API usage
- Test upgrade in dev → UAT environments first
- Update all add-ons (CoreDNS, kube-proxy, VPC CNI) to compatible versions

Phase 2: Control Plane Upgrade

```
resource "aws_eks_cluster" "main" {
  version = "1.30" # Increment by one minor version at a time
}
```

- Control plane upgrade is managed by AWS — zero downtime, takes ~25 minutes
- API server remains available throughout

Phase 3: Node Group Rolling Update

```
resource "aws_eks_node_group" "main" {
  version = aws_eks_cluster.main.version

  update_config {
    max_unavailable_percentage = 25 # Roll 25% of nodes at a time
  }
}
```

- Pods are drained gracefully with PodDisruptionBudgets
- New nodes launch with updated AMI → old nodes drain → terminate

PodDisruptionBudget (critical for banking):

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: payment-service-pdb
spec:
  minAvailable: 2 # Always keep 2 payment pods running
  selector:
    matchLabels:
      app: payment-service
```

Q24. How do you implement network policies on EKS for a banking application?

Answer:

Default Kubernetes networking allows **all pods to communicate** — unacceptable for banking. I implement **default-deny with explicit allow**:

```
# Default deny ALL traffic in payments namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: payments
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]

---
# Allow payment-service to receive traffic only from API gateway
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-ingress
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: channels
          podSelector:
            matchLabels:
              app: api-gateway
  ports:
    - port: 8443
      protocol: TCP

---
# Allow payment-service egress only to database and KMS endpoint
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-egress
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.100.0/24 # RDS subnet CIDR
  ports:
```

```
- port: 5432
- to:
  - ipBlock:
      cidr: 10.0.200.0/24 # VPC endpoint CIDR
  ports:
    - port: 443 # KMS, Secrets Manager
```

Requires **Calico CNI** (AWS VPC CNI supports network policies natively since EKS 1.25+).

Q25. How do you handle pod security in a banking EKS cluster?

Answer:

I enforce **Pod Security Standards** at the namespace level:

```
apiVersion: v1
kind: Namespace
metadata:
  name: payments
  labels:
    # Enforce restricted profile – strictest security
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

What "restricted" enforces:

- No privileged containers
- No privilege escalation (**allowPrivilegeEscalation: false**)
- Non-root user required (**runAsNonRoot: true**)
- Read-only root filesystem
- No host namespace sharing (hostNetwork, hostPID, hostIPC)
- Restricted volume types (no hostPath)
- Seccomp profile required

Additional Banking Controls:

```
securityContext:
  runAsNonRoot: true
```

```
runAsUser: 1000
readOnlyRootFilesystem: true
allowPrivilegeEscalation: false
capabilities:
  drop: ["ALL"]
seccompProfile:
  type: RuntimeDefault
```
